

Внутреннее файловое хранилище MK61s mini

Версия документа: 19.07.2026

MK61s mini хранит программы, тексты, состояния, шрифты и каталоги во внешней SPI NOR flash. Постоянным источником истины является собственная файловая система C5 (program_store). USB Mass Storage показывает её как динамически сформированный FAT12-диск (virtual_fat). Готового FAT-образа во flash нет.

C5 намеренно не читает C1--C4. При первом запуске этой версии старое хранилище считается неразмеченным, реальная ёмкость flash измеряется, после чего создаётся новая C5. Миграции и восстановления старых файлов нет.

Эталонная реализация находится в:

- code/spi_nor_flash.cpp -- JEDEC/SFDP и низкоуровневый SPI NOR;
- code/flash_capacity_probe.hpp -- проверка физической границы;
- code/storage_geometry.cpp -- расчёт C5 и FAT12;
- code/program_store.cpp -- каталог, WAL, данные, GC и USB staging;
- code/storage_path.cpp -- единый разбор абсолютных и относительных путей;
- code/virtual_fat.cpp -- FAT12-проекция и импорт изменений хоста.

Основные свойства

Параметр	C5
Физический erase-сектор	4096 байт
Логический сектор USB	512 байт
Поддерживаемая ёмкость SPI NOR	128 КиБ--128 МиБ
Максимальная логическая длина файла	1536 байт
Максимальная длина basename	31 байт UTF-8
Максимальная глубина каталогов	32
Копий FAT	2
Максимум кластеров данных FAT12	4084
Зарезервировано под настройки	1 erase-сектор
Доступно журналу настроек	4080 байт
Одновременно staged USB-блоков	384 по 512 байт на flash от 2 МиБ; 96 на 512 КиБ

Квота объектов больше не фиксирована. Она вычисляется из измеренной ёмкости, физического worst case и границы FAT12. На двух целевых микросхемах:

Физическая flash	Узлов C5	Размер FAT-кластера	Секторов FAT12
512 КиБ	192	2 КиБ	832
16 МиБ, W25Q128	4084	4 КиБ	32729

Узел -- файл, каталог или скрытый extent большого каталога. Поэтому 4084 -- предельная квота всего тома, а не обещание 4084 файлов непосредственно в корне. Каталог сам занимает один узел. Дополнительный extent расходуется, когда его записи не помещаются в один FAT-кластер.

Лимит файла оставлен равным 1536 байтам намеренно: приоритет отдан количеству файлов и малому расходу RAM редакторами. FOCAL, Tiny BASIC, M61, текстовый и шрифтовый редакторы используют ту же квоту.

Определение физической ёмкости

Быстрый путь при обычной загрузке

Драйвер читает JEDEC ID командой 0x9F, затем заголовок и Basic Flash Parameter Table SFDP командой 0x5A. Значения сохраняются как заявленная геометрия и помогают выбрать протокол команд, но не ограничивают поиск: они не считаются доказательством физической ёмкости и могут быть ошибочными в обе стороны.

Если одна из двух C5 locator-копий валидна, загрузка является read-only:

- 1 Проверяются сигнатура, версия, committed-state и CRC32 locator.
- 2 Геометрия полностью пересчитывается и сравнивается с сохранёнными полями.
- 3 Сравниваются JEDEC ID и заявленная JEDEC/SFDP-граница.
- 4 На измеренном дальнем конце читается guard сектора настроек.
- 5 Драйвер переключается на сохранённую измеренную ёмкость.

Ни erase, ни тестовая запись на штатной загрузке не выполняются. Это сохраняет ресурс flash. Завышенная маркировка, однажды измеренная как меньшая физическая ёмкость, также не вызывает повторное форматирование при каждом включении.

Деструктивный пробник неразмеченной flash

Пробник запускается только тогда, когда нет валидного C5 locator. Содержимое такой микросхемы всё равно будет заменено новой C5, поэтому тестовые секторы не копируются и не восстанавливаются.

Стандартные ёмкости SPI NOR являются степенями двойки. Независимо от заявленной ёмкости по показателям степени от 17 до 27 выполняется настоящий двоичный поиск максимальной рабочей границы:

128 КиБ, 256 КиБ, 512 КиБ, ... 64 МиБ, 128 МиБ

Для проверки candidate используются два сектора:

```
lower = candidate / 2 - 4096
upper = candidate - 4096
```

Оба сектора стираются, затем в них записываются разные комплементарные 32-байтные метки. После обеих записей обе метки читаются и сравниваются. Если старшие адресные биты отсутствуют и upper зеркалит lower, вторая запись потребует невозможного для NOR перехода 0 -> 1 либо финальное сравнение увидит alias. Ошибка команды, verify или таймаут также означает нерабочую границу.

Для диапазона 128 КиБ--128 МиБ требуется не более четырёх проверок candidate, то есть не более восьми основных erase-операций. При проверке границ выше 16 МиБ дополнительно защищаются два низких shadow-сектора. Это обнаруживает трёхбайтную микросхему, которая проигнорировала EN4B и приняла четвёртый байт адреса за данные. Таких защитных стираний не более шести за весь поиск; общий худший случай -- 14, а для микросхем до 16 МиБ их нет.

Стирания и page-program одной SPI NOR нельзя выполнять параллельно: кристалл выставляет BUSY. Реализация не делает лишнего промежуточного чтения -- обе метки программируются последовательно, а затем проверяются одним этапом чтения.

Поиск всегда включает 128 МиБ, поэтому заниженные JEDEC/SFDP не оставляют неиспользованную физическую ёмкость. Если не проходит даже 128 КиБ, C5 не создаётся.

Адресация больше 16 МиБ

Для микросхем с четырёхбайтным адресом драйвер разбирает optional SFDP 4-Byte Address Instruction Table (4BAIT). Если таблица одновременно объявляет обычные read, page-program и выбранный 4-КиБ erase type, используются stateless-команды 13h, 12h и указанный таблицей erase opcode. В остальных случаях драйвер разбирает BFPT DWORD 16 и использует B7h/E9h, в том числе вариант с обязательным WREN. Неизвестный способ переключения не угадывается: граница выше 16 МиБ в таком случае не принимается.

SFDP также задаёт размер program-page. Драйвер никогда не отправляет за одну операцию больше 256 байт и уменьшает порцию для микросхем с меньшей страницей, исключая page-wgap.

Горячий путь SPI использует буферные transfer-операции STM32 HAL: сектор читается одним вызовом, page-program -- одним вызовом на физическую страницу, а verify 512 байт -- одной READ-командой и одной порцией сравнения. Побайтового входа в HAL для каждого из 512 байт нет. Постоянная RAM при этом не расходуется; для verify используется ограниченный 512-байтный stack buffer.

Что именно гарантирует быстрый тест

Разреженный пробник доказывает адресуемый диапазон, отсутствие зеркалирования на проверяемых границах и работоспособность последнего сектора, отданного настройкам. Он намеренно не является полным тестом здоровья каждого сектора: такой тест потребовал бы стереть и записать всю микросхему и занял бы минуты.

Все реальные записи C5, включая locator, каталоги, WAL, данные и настройки, проверяются чтением. Поэтому отдельный изношенный сектор приводит к явному отказу операции, а не к подтверждению повреждённых данных; неиспользованные data-секторы проверяются лениво при первом обращении.

Единицы getCapacity()

getCapacity() возвращает байты. Старый диагностический код ошибочно печатал после числа суффикс К:

```
16777216 = 16 МиБ, W25Q128, а не 16777216 КиБ
524288   = 512 КиБ, а не 524288 КиБ
```

Новый вывод явно пишет bytes и отдельно показывает заявленную верхнюю и измеренную C5-ёмкость.

Динамическая карта SPI NOR

Все числа ниже -- номера 4096-байтных erase-секторов. Ни один сектор после измеренной границы не используется.

```
0          locator A
1          locator B
2 ..      catalog bank A
          catalog bank B
          packed data log
... settings-stage  USB staging log
capacity/4096 - 1   settings journal + C5 guard
```

Один catalog bank состоит из:

```
1 sector header + ceil(max_nodes * 20 / 4096) table sectors + 2 WAL sectors
```

Размер staging динамический: на flash от 2 МиБ это 65 секторов (64 рабочих и один резерв атомарной компактизации), на 512 КиБ -- 17, а ниже -- 4--8. Оставшиеся секторы между каталогами и staging являются журналом данных. Один сектор данных и ещё один сектор удерживаются для append/GC progress.

Примеры фактической карты:

Поле	512 КиБ	16 МиБ
Физических секторов	128	4096
Секторов таблицы в bank	1	20
Размер одного bank	4	23
Первый сектор данных	10	48
Секторов данных	100	3982
Первый сектор staging	110	4030
Секторов staging	17	65
Сектор настроек	127	4095

Таким образом, неразмеченного хвоста нет: каждый физический сектор до измеренной границы относится к locator, каталогу, данным, staging или настройкам. Области данных стираются лениво, только перед первым использованием.

Locator и смена микросхемы

Две независимые locator-копии C5FS находятся в секторах 0 и 1. Каждая хранит:

- версию формата и committed-state;
- измеренную ёмкость и format epoch;
- все рассчитанные границы областей;
- число узлов и FAT-геометрию;
- заявленную JEDEC/SFDP-границу и JEDEC ID;
- CRC32 всей записи.

На дальнем конце находится независимый C5SG guard с измеренной ёмкостью и CRC32. Он обнаруживает уменьшение flash, перенос образа на другую ёмкость и простое адресное зеркалирование.

Изменение JEDEC ID, заявленной JEDEC/SFDP-границы, измеренной ёмкости или C5 capacity guard означает замену/несоответствие микросхемы. Тогда граница измеряется заново и создаётся новая C5. Настройки при этом стираются, потому что их физический сектор мог переместиться.

Если отличается только рассчитанная новой прошивкой C5/FAT-геометрия, но совпадают chip ID, заявленная и измеренная ёмкость, дальний settings-sector и его CRC guard, деструктивный пробник не запускается: файловая часть создаётся заново, а 4080 байт настроек сохраняются. Повреждение только обоих catalog bank при валидной геометрии также форматирует файловую часть с сохранением settings-sector.

Каталог C5

ID узла стабилен и имеет ширину 16 бит. Таблица содержит по 20 байт на ID:

- 32-битный адрес текущей записи;
- длину данных и физическую длину записи;
- parent ID;
- first-child, next-sibling и prev-sibling;
- hash имени;
- kind/type и flags.

Два copy-on-write bank содержат полный checkpoint. Новый checkpoint сначала стирается и заполняется в неактивном bank, защищается CRC32 и только последней операцией получает ACTIVE. Старый bank до этого остаётся валидным.

Между checkpoint находятся WAL-записи w5 по 256 байт. Одна запись атомарно публикует metadata и до девяти изменённых inode; commit-state программируется последним байтом. Два WAL-сектора дают 32 транзакции между checkpoint. В RAM держится не весь каталог, а 512-байтный cache таблицы и до 96 overlay-изменений. Overlay и индекс USB staging представлены отдельными плотными массивами полей: выравнивание структур не оставляет скрытых хвостов. Свободный ID ищется от запомненной подсказки, поэтому последовательное создание тысяч файлов не превращается в квадратичный просмотр начала таблицы. Когда WAL или overlay заполняется, создаётся новый checkpoint.

CRC защищает header, таблицу, WAL и каждую запись данных. При частичной записи после сбоя используется последняя полностью committed версия.

Физический журнал данных и отсутствие хвостов по 4 КиБ

Сектор данных имеет 16-байтный header C5D0. Далее без кластерного выравнивания упаковываются записи:

```
16-byte record header + UTF-8 basename + raw payload
```

Запись не пересекает erase-сектор, но файл не получает отдельный 4-КиБ сектор. Максимальная запись занимает $16 + 31 + 1536 = 1583$ байта, поэтому даже две максимальные записи гарантированно

помещаются в один сектор. Маленькие записи упаковываются плотнее. Неиспользуемым остаётся только хвост, в который уже не помещается следующая конкретная запись; фиксированной потери 4 КиБ на файл нет.

Обновление и rename дописывают новую запись. После WAL commit inode начинает указывать на неё; старая версия становится мусором. Повторная запись полностью идентичных данных является по-ор и не расходует ресурс flash.

GC использует отдельный reserve-sector. Поиск свободного или наиболее выгодного victim идёт окнами по 32 сектора: таблица inode читается один раз на окно, а не заново для каждого сектора. Живые записи victim копируются в reserve и их новые адреса публикуются атомарными WAL-пакетами до девяти inode. Только после публикации всех адресов victim стирается и становится новым резервом. Потеря питания оставляет старые либо уже committed новые ссылки, но не промежуточный inode.

Каталоги и имена

C5 поддерживает создаваемые пользователем каталоги с произвольной структурой. Каталог хранит head двусвязного списка непосредственных детей, поэтому его просмотр не требует читать или сортировать всё оглавление flash. Максимальная глубина -- 32. Проверка учитывает всё поддерево и не позволяет обойти лимит переносом глубокого каталога.

Внутреннее имя -- валидный UTF-8 basename длиной 1--31 байт. Запрещены:

- управляющие байты и FAT-символы < > : " / \ \ | ? *;
- . и ..;
- завершающие пробел или точка;
- DOS device names CON, PRN, AUX, NUL, COM1--COM9, LPT1--LPT9, в том числе перед расширением.

В пределах каталога не допускаются имена, которые совпадают в FAT-проекции без учёта регистра. Сравнение выполняет Unicode simple case folding для ASCII, латиницы, греческого, кириллицы и ряда других регистровых диапазонов; письменности без регистра сравниваются без преобразования. Например, Каталог конфликтует с каталог, каталог demo.m61 -- с MK61-файлом basename demo, a Report.txt -- с report.txt.

Файлы получают расширение по типу:

Тип	Расширение USB
MK61	.m61
FOCAL	.foc
Tiny BASIC	.tbi
TEXT	.txt
MK61 state	.state.txt
FONT	.fmk

Проводник, редакторы и общий файловый диалог

Проводник работает с непосредственными детьми текущего каталога и не строит в RAM плоский список всего диска. Короткое ОК входит в папку, загружает M61 либо запускает файл соответствующего языка. Долгое ОК открывает действия:

- просмотр или запуск/загрузка файла;
- редактирование FOCAL и Tiny BASIC;
- создание произвольной папки;
- переименование;
- перенос в выбранный каталог;
- удаление файла, пустой папки или подтверждённое рекурсивное удаление дерева.

Имена до 31 байта UTF-8 отображаются и прокручиваются по Unicode-символам; курсор и Сх в редакторе имени не разрезают многобайтную последовательность. Имена и строка поиска используют ту же раскладку, что языковые редакторы: цифры без префикса вводят цифры, К + цифровая клавиша включает SMS-ввод, ПП вводит пробел, F + цифровая клавиша - спецсимвол. Сх выполняет забор, а F + Сх очищает поле целиком.

FOCAL, Tiny BASIC и M61 не содержат собственных плоских браузеров. Они вызывают общий API `program_store_choose_file()`, `program_store_choose_directory()` и `program_store_choose_save_target()`. В диалоге можно переходить по дереву и создавать папки; редактор сохраняет файл в выбранный каталог. В меню Разработка -> Файлы МК-61 доступны отдельные операции открытия и сохранения текущей программы. Числовые клавиши LOAD/SAVE и терминальные slots 0 . . 99 сохранены как быстрый интерфейс к числовым M61-файлам в корне.

Пути и терминал

Единый path API принимает / и \, абсолютные и относительные пути, повторные разделители, . и . . . Путь с пробелами можно заключить целиком в одинарные или двойные кавычки. Расширение задаёт тип файла; имя без расширения разрешается только при единственном совпадении basename, иначе возвращается ошибка ambiguity.

Интерактивный терминал хранит текущий каталог и показывает его в prompt. Основные команды:

```
pwd
cd [path]
ls [path]                # aliases: dir, fsls
mkdir [-p] path
mv source destination
rm [-r] path             # aliases: del, fsm
rmdir path
df                       # alias: fsstat
save path[.m61]
load path[.m61]
open path
run path
```

mv не перезаписывает существующий объект. rm удаляет файл, rm -r -- всё дерево, rmdir -- только пустую папку. df выводит измеренную ёмкость flash, занятые/свободные узлы, число файлов, каталогов и extent, виртуальный размер кластера и резерв настроек. Все изменяющие дерево команды запрещены внутри M61-сценариев явным allowlist.

Динамический FAT12

USB-хост видит стандартный FAT12 superfloppy без partition table:

- 512 bytes per sector;
- 1 reserved sector;
- 2 FAT;
- media descriptor 0xF8;
- фиксированный root directory на 512 entries;
- volume label MK61S C5;
- строго не больше 4084 data clusters.

Порог FAT12 определяется числом кластеров, а не экспериментальной CHS-формой дискеты. Геометрии 2M и других floppy-драйверов подтверждают, что BPB может быть нестандартным, но для современных USB Mass Storage C5 сохраняет обычную семантику FAT12 и не зависит от CHS.

Размер кластера выбирается по измеренной flash:

Flash	Секторов на кластер	Кластер
128 КиБ--8 МиБ	4	2 КиБ
16 МиБ	8	4 КиБ

Flash	Секторов на кластер	Кластер
32 МиБ	16	8 КиБ
64 МиБ	32	16 КиБ
128 МиБ	64	32 КиБ

Это только виртуальные FAT-кластеры. Кластер 4 КиБ не означает физическую потерю 4 КиБ на файл: payload по-прежнему упакован в журнал C5.

ID узла `id` всегда показывается как `FAT cluster id + 2`. Поэтому кластеры файлов и каталогов стабильны при удалении соседних объектов. Файл длиной до 1536 байт занимает один виртуальный кластер. Каталог использует свой кластер; если `directory entries` не помещаются, C5 выделяет скрытые extent-ID и связывает их обычной FAT12-цепочкой.

Root directory остаётся фиксированной FAT12-областью на 512 entries (16 КиБ). Четыре заняты системно: volume label и LFN/short-записи маркера `.metadata_never_index`, запрещающего Spotlight индексировать диск. Для пользовательских объектов остаётся 508 entries: от 101 объекта при самом длинном имени (четыре LFN + short) до 254 объектов, чьё видимое имя занимает не более 13 UTF-16 units (одна LFN + short). C5 считает entries точно и атомарно отклоняет `create/move/rename`, который переполнил бы root; уже созданные объекты при этом не меняются. Полная квота 4084 узла доступна через произвольные подкаталоги, которые растут обычными FAT-цепочками. Компактный root принципиален для скорости: macOS может переписывать значительную часть фиксированного root при каждом создании файла.

Подкаталоги содержат стандартные `.` и `...`. Short alias строится из стабильного ID (`Fxxxx` для файла, `Dxxxx` для каталога), поэтому не коллидирует после `rename`. Пользователь всегда видит LFN.

USB staging и sync

Хост может записывать FAT, каталог и data sectors в любом порядке. Поэтому каждый принятый 512-байтный блок сначала попадает в persistent staging log, а не в RAM и не сразу в C5-файл.

Один staging-сектор содержит 16-байтный header и семь записей вида:

```
16-byte record header + 512-byte payload
```

Header хранит LBA-key, поколение, state и CRC32. Сначала записываются header и payload, последним байтом -- ACTIVE. На обычной flash до 384 последних live-блоков индексируются в RAM одним packed-массивом `LBA + record-ref`; 65-й сектор служит резервом атомарной компактизации. На 512-КиБ варианте 16 рабочих секторов дают до 96 live-блоков, а 17-й является резервом.

Finder создаёт 4-КиБ AppleDouble-sidecar `._`имя для extended attributes. Такой временный файл может иметь то же расширение `.m61`, `.foc` и т. п., но не является программой. Двухпроходный импорт распознаёт префикс `._`, не включает sidecar в C5 и не позволяет ему сорвать `commit` настоящего файла. Неизвестные расширения также игнорируются до проверки C5-квоты размера. После успешного sync их staged-блоки отбрасываются.

При SYNC virtual FAT выполняет два прохода:

- 1 Проверяет FAT-цепочки, LFN, уникальность cluster-ID, глубину, размеры и directory tree без публикации payload.
- 2 Создаёт/перемещает каталоги, собирает staged data и атомарно публикует файлы через C5 WAL.

Циклы FAT, выход за диапазон, повторное использование cluster-ID, неверный LFN, файл больше 1536 байт и неподдерживаемая глубина отклоняются. Staged blocks при ошибке не теряются и могут быть повторно синхронизированы или явно отброшены. После `reset committed staging` восстанавливается по поколениям и CRC.

Настройки

Последний физический erase-сектор всегда зарезервирован под настройки:

```
offset 0..4079      settings journal
offset 4080..4095   C5SG capacity guard
```

Обычная очистка файловой системы и восстановление каталога не стирают журнал настроек. API `erase-settings` стирает сектор целиком и немедленно восстанавливает `guard`. При смене измеренной ёмкости сектор переезжает, поэтому создаётся новый пустой журнал.

Гарантии и границы

- C5 не обещает совместимость с C1--C4 и не пытается их сканировать.
- Валидная C5 загружается без тестовых `erase/write`.
- JEDEC/SFDP -- подсказка; окончательная граница неразмеченной flash подтверждается записью и `alias-aware verify`.
- Полный файл и отдельная операция каталога публикуются атомарно через WAL.
- FAT12 является представлением объектов, а не побайтовой копией SPI NOR.
- Свободные байты FAT не равны числу свободных физических байтов журнала; рабочая квота выражена числом свободных `node-ID`.
- CRC32 обнаруживает случайные повреждения, но не является криптографической защитой.
- При физической неисправности обеих C5 `locator`-копий содержимое считается неразмеченным и форматируется; эвристического восстановления нет.